

Mixed Boolean-Arithmetic Obfuscation

What We Build, What We Break, and What We Can't

Arnau Gàmez i Montolio

REcon - June 21, 2026 - Montreal

About

Arnau Gàmez i Montolio

Hacker, Reverse Engineer, Mathematician

- Founder, Principal Consultant & Trainer @ Fura Labs.
- **Principal Research Engineer/Scientist @ Hex-Rays.**
- PhD researcher @ City St George's, University of London.

Views are my own. Thanks to Hex-Rays for the freedom to share research, opinions, and rants.

A note on collaboration and credit

Many of the ideas you'll hear here are not solely mine.

A lot of the thinking in this talk has been shaped by years of conversations and experiments with my friend **Cody Dresel** (cpl0.com).

He rarely takes credit, but a lot of public work in this area has privately relied on his guidance.

Also shoutout to my PhD supervisors **Martin Brain** and **Jacob Howe**.

$\mathbb{Z}/2^w\mathbb{Z} \equiv w\text{-bit values}$

[whatever] modulo 2^w $\equiv$ values/expressions/computations are considered in w bits

What is MBA?

Algebraic expressions that **mix arithmetic and bitwise operators**.

Usually arithmetic $\{+, -, \times\}$ and bitwise $\{\wedge, \vee, \oplus, \neg\}$.

The actual set depends on who you ask. **More on that later.**

Operation	Math	Code
AND	$\wedge$	<code>&</code>
OR	$\vee$	<code> </code>
XOR	$\oplus$	<code>^</code>
NOT	$\neg$	<code>~</code>

Note: I will use interchangeably the terms *boolean*, *bitwise* and *logic* operators.

Obfuscation with MBA

From a *simple* expression E_1 , produce a *complex* E_2 that computes the same mapping.

Example: obfuscated form of 8-bit addition $x + y$.

$$\begin{aligned} x + y = & ((((((18 + (75 \cdot ((74 + ((x \vee (y \wedge y)) \cdot 70)) + ((\neg((\neg x \vee \neg x) \wedge \neg y) \cdot 160) \cdot (2 \cdot (x \vee (y \wedge y)))))))))) + \\ & ((16 \cdot ((74 + ((x \vee (y \vee y)) \cdot 70)) + (((x \vee (x \vee y)) \cdot 160) \cdot (2 \cdot (x \vee (y \vee y)))))) \cdot ((74 + (\neg((\neg x \wedge \neg y) \wedge \neg y) \cdot \\ & 70)) + ((\neg((\neg x \vee y) \wedge \neg y) \cdot 160) \cdot (2 \cdot \neg((\neg x \vee \neg x) \wedge \neg y)))))) + 97) + (249 \cdot ((167 + ((x \oplus (y \wedge y)) \cdot 87)) + \\ & ((\neg(\neg(\neg x \vee \neg x) \oplus \neg y) \cdot 176) \cdot (255 \cdot \neg(\neg(\neg x \wedge \neg x) \oplus \neg y)))))) + ((48 \cdot ((167 + (\neg(\neg(\neg x \vee \neg x) \oplus \neg y) \cdot \\ & 87)) + (((x \oplus (y \wedge y)) \cdot 176) \cdot (255 \cdot (x \oplus (y \vee y)))))) \cdot ((167 + ((x \oplus (y \wedge y)) \cdot 87)) + ((\neg(\neg(\neg x \wedge \neg x) \oplus \\ & \neg y) \cdot 176) \cdot (255 \cdot (x \oplus (y \vee y)))))) \end{aligned}$$

Example: obfuscated form of 8-bit constant 123.

$$\begin{aligned} 123 = & 195 + 97 \cdot x + 159 \cdot y + 194 \cdot \neg(x \vee \neg y) + 159 \cdot (x \oplus y) + (163 + x + 255 \cdot y + 2 \cdot \neg(x \vee \neg y) + 255 \cdot \\ & (x \oplus y)) \cdot (232 + 248 \cdot x + 8 \cdot y + 240 \cdot \neg(x \vee \neg y) + 8 \cdot (x \oplus y)) \end{aligned}$$

Agenda

BUILD

1. "MBA" is many problems, not one.
2. Syntax vs. semantics.

BREAK

3. We break what we build.
4. Datasets are biased, but necessary.
5. MBA in the wild is embedded.
6. "Simplified" is a measurement choice: metric and reference.

CAN'T

7. Normal forms unlock the foundations.
8. Toward algebraic synthesis.

BUILD

1. "MBA" is many problems, not one.

MBwAt?

You say

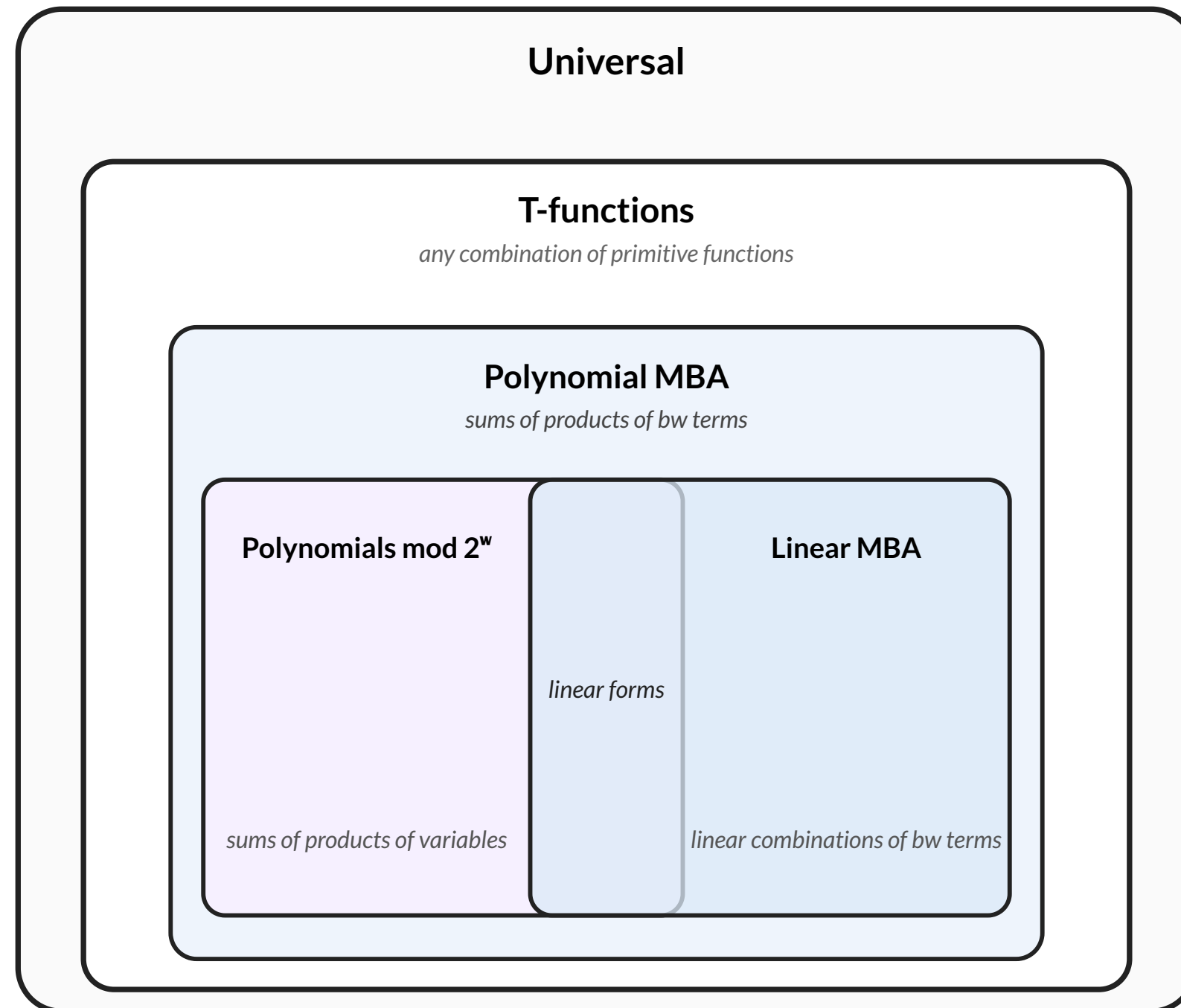
I "broke" MBA

I have no idea what object you're talking about

- linear vs. polynomial vs. (*whichever definition of*) non-polynomial?
- arbitrary constants allowed in bitwise sub-expressions, or not?
- do you account for right shift? rotations? or just "the usual" operators?

The "MBA problem" is many problems. **Which slice are you talking about?**

Where it lives in the lattice



✓ **Common alphabet** $\{+, -, \times, \wedge, \vee, \oplus, \neg\}$

✗ **Add right shift or rotate:** out of T-functions. Different territory.

? **Arbitrary constants:** Most constructions exclude them from the bitwise components of linear and polynomial MBA.

What are we even talking about?

A naming rant

Terms I'd retire from the literature:

- **"Semi-linear MBA"** just means *linear MBA with constants inside the bitwise sub-expressions*. Say that. The label tells me nothing.
- **"Non-polynomial MBA"** is defined differently by different papers (some include right shifts, some don't; some carve out sub-cases without naming them). If you mean "anything not polynomial MBA", say so. If you want a specific subset, define it properly and name it.
- **"Binary polynomials"** (my past sin too) suggests $\{0, 1\}$ coefficients.
I'll say *polynomials mod 2^w* (or *polynomials over bit-vectors*) from here on.

Naming a slice is fine. Naming it carelessly is what makes the lattice harder to navigate.

Briefly: what's a T-function?

A function where **each output bit depends only on input bits at or below it.**

output bit	b_0	b_1	b_2	$\dots$
depends on input bits	b_0	b_0, b_1	b_0, b_1, b_2	$\dots$

The common MBA operators $\{+, -, \times, \wedge, \vee, \oplus, \neg\}$ are **primitive** T-functions. **Left shift** ($<<$) also comes for free (multiplication by a power of 2).

Any combination of them is also a T-function: the common-alphabet MBA stays inside this class.

Right shifts and rotations break this: bit i can depend on higher bits.

BUILD

2. Syntax vs. semantics.

Expressions vs. mappings

An **expression** is syntax: the written form.

$$f(x, y) = x + y \pmod{2^8}, \quad g(x, y) = (x \oplus y) + 2(x \wedge y) \pmod{2^8}$$

A **mapping** is semantics: the meaning of the expression.

x	y	$f(x, y)$	$g(x, y)$
1	2	3	3
20	30	50	50
250	10	4	4

Different expressions can compute the same mapping.

Grammars vs. function classes

Grammar (syntax) → generates **expressions**

- alphabet of symbols (variables, constants, operators)
- rules for combining them

Function class (semantics) → the **mappings** a grammar can express

- the set of all meanings denoted by some expression of the grammar

Two grammars can represent...

- the same function class, or
- different function classes.

The first is less obvious.

Example: 8-bit constants, one variable x . The following define the same function class.

- $\{+, -, \times\}$: $3 + 4x - 7x^2 \pmod{2^8}$
- $\{+, -x, \times\}$: $3 + 4x + (-7)x^2 \pmod{2^8}$
- $\{+, \times\}$: $3 + 4x + 249x^2 \pmod{2^8}$

Which case you're in, for two given grammars: we mostly don't know in general.

Case study: univariate polynomials mod 2^w

Over $\mathbb{Z}$ or $\mathbb{R}$

Distinct polynomial expressions denote distinct mappings. **1-to-1** between syntax and semantics.

Over $\mathbb{Z}/2^w\mathbb{Z}$

They don't in general.

$$f(x) = 2x \pmod{2^2}, \quad g(x) = 2x^2 \pmod{2^2}$$

$$f(0) = g(0) = 0 \quad f(1) = g(1) = 2 \quad f(2) = g(2) = 0 \quad f(3) = g(3) = 2$$

How many polynomial mappings are there?

Pigeonhole principle.

- Mappings over $\mathbb{Z}/2^w\mathbb{Z}$ are finite: $(2^w)^{2^w}$.
- Expressions are unbounded (can be infinitely large).

Thus, distinct expressions *must* collide on mappings.

Naive count

Fix degree d , take $P(x) = a_0 + a_1x + \dots + a_dx^d$, each $a_i \in \mathbb{Z}/2^w\mathbb{Z}$.

Count: $(2^w)^{d+1}$. For arbitrarily large d , exceeds $(2^w)^{2^w}$. **Absurd.**

Actual count

Degree at most $d_w - 1$ ($d_w = w + 2$ for w a power of two).

Each a_j admits $2^{w-\nu_2(j!)}$ values.

$$\prod_{j=0}^{d_w-1} 2^{w-\nu_2(j!)}$$

$\nu_2(j!)$ is the 2-adic valuation; how many 2s divide $j!$.

There's a closed-form count, far smaller than naive counting suggested. No grammar shows it.

The gap, on 8 bits

Total mappings

$$(2^8)^{2^8} = 2^{2048}$$

$\approx$ 617-digit number

Polynomial-expressible

$$\prod_{j=0}^9 2^{8-\nu_2(j!)} = 2^{50}$$

$$= 1,125,899,906,842,624$$

A vanishing fraction (2^{50} out of 2^{2048}). Worse on higher bitsizes.

This applies to any bitsize and any number of variables.

Efficient Normalized Reduction and Generation of Equivalent Multivariate Binary Polynomials. Gàmez-Montolio, Florit, Brain, Howe. BAR 2024. arnaugamez.com/assets/papers/bar2024.pdf

BREAK

3. We break what we build.

Build / break #1: linear MBA identities

Build. Sample truth tables, solve linear systems over the booleans, insert any of the resulting identities.

Break. Same boolean linear algebra, run in the opposite direction.

Build / break #2: catalogs of rewrite rules

Build. Pick rules from public catalogs: *Hacker's Delight* (Warren), the appendix of Ninon Eyrolles' PhD thesis. Apply iteratively.

Break. Match the same rules in the opposite direction.

Build / break #3: same operator alphabet

Build. Stick to grammars most obfuscators use. Generate variants in this grammar.

Break. Synthesize the simplest expression in **the same grammar** that matches the observed input-output behaviour.

For years, commercial software-protection products have just copy-pasted these rules or gone with purely linear rewrites.

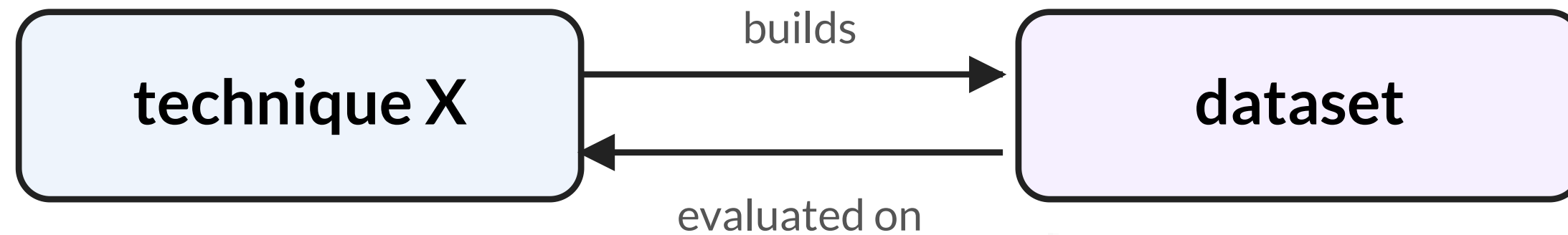
Many still do today.

BREAK

4. Datasets are biased, but necessary.

Datasets and bias

Techniques evaluated against the constructions they were built for.



This measures **closure within a family**, not capability beyond it.

Datasets are still **necessary** (reproducibility, comparison, regression detection).

The gap is just rarely acknowledged.

BREAK

5. MBA in the wild is embedded.

The simplifier may not get a clean expression

MBA in real binaries is **embedded inside other deobfuscation problems**.

- MBA destructured by the compiler optimizer.
- MBA mangled by lifters / decompilers.
- MBA split across basic blocks (intentional).
- MBA split across functions (intentional).
- MBA interleaved with virtualization, opaque predicates, CFF (intentional).

Just simplify works only **locally**, on the fragments you can see. That's often enough against bad obfuscators (many shipping products fall here). It is **not enough in general**.

The deobfuscation loop

MBA simplification is **one stage** in a back-and-forth between many deobfuscation steps, e.g.:

- partial extraction of a sub-expression, simplify, push insight back into the IR.
- defeat an opaque predicate so a branch resolves, then re-do the extraction.
- normalize control-flow (CFF removal) so what was an MBA across many blocks lives in one.
- repeat on a different sub-expression that the previous simplification exposed.

BREAK

6. "Simplified" is a measurement choice: metric and reference.

A polynomial over $\mathbb{Z}/2^8\mathbb{Z}$:

$$\begin{aligned} f(x) = & x^{100} + 156x^{99} + 86x^{98} + 92x^{97} + 73x^{96} + 32x^{95} + 176x^{94} + 160x^{93} + 28x^{92} + 112x^{91} + 56x^{90} + \\ & 240x^{89} + 140x^{88} + 96x^{87} + 80x^{86} + 224x^{85} + 186x^{84} + 120x^{83} + 172x^{82} + 120x^{81} + 26x^{80} + 96x^{79} + 144x^{78} + \\ & 224x^{77} + 156x^{76} + 48x^{75} + 88x^{74} + 176x^{73} + 108x^{72} + 32x^{71} + 112x^{70} + 160x^{69} + 215x^{68} + 100x^{67} + 74x^{66} + \\ & 36x^{65} + 63x^{64} + 64x^{63} + 224x^{62} + 64x^{61} + 248x^{60} + 224x^{59} + 112x^{58} + 224x^{57} + 88x^{56} + 192x^{55} + 32x^{54} + \\ & 192x^{53} + 204x^{52} + 144x^{51} + 104x^{50} + 144x^{49} + 204x^{48} + 192x^{47} + 32x^{46} + 192x^{45} + 88x^{44} + 224x^{43} + \\ & 112x^{42} + 224x^{41} + 248x^{40} + 64x^{39} + 224x^{38} + 64x^{37} + 63x^{36} + 36x^{35} + 74x^{34} + 100x^{33} + 215x^{32} + 160x^{31} + \\ & 112x^{30} + 32x^{29} + 108x^{28} + 176x^{27} + 88x^{26} + 48x^{25} + 156x^{24} + 224x^{23} + 144x^{22} + 96x^{21} + 26x^{20} + 120x^{19} + \\ & 172x^{18} + 120x^{17} + 186x^{16} + 224x^{15} + 80x^{14} + 96x^{13} + 140x^{12} + 240x^{11} + 56x^{10} + 112x^9 + 28x^8 + 160x^7 + \\ & 176x^6 + 32x^5 + 73x^4 + 92x^3 + 86x^2 + 156x + 1 \end{aligned}$$

Normalize it down to the unique degree-bounded representative:

$$f(x) = x^9 + 220x^8 + 38x^7 + 248x^6 + 53x^5 + 229x^4 + 200x^3 + 198x^2 + 92x + 1$$

Surely *this* is the "simplified" form. **Right?**

The same function is just:

$$f(x) = (x - 1)^{100}$$

Three different expressions, one mapping over $\mathbb{Z}/2^8\mathbb{Z}$. **Which one is "simplest"?**

And it's not just polynomials: the same multi-representation problem shows up on compact vs. boolean-basis representations of linear MBA.

The metric

There is no single way to measure "simpler". Each choice gives a different answer:

- **Degree.** Normalized wins.
- **Sparsity / symbol count.** Compact wins (if we allow exponentiation in the grammar; rarely done, because high degree breaks algebraic reasoning).
- **Tree depth, IR instructions, runtime cost, human readability, ...**

Each metric optimizes a different thing. No universal answer. Always a compromise.

The compromise itself is rarely stated.

The reference

Simpler than what?

A metric tells you how to *measure* simplicity. You still need a **reference** to compare against:

- the obfuscated input?
- the ground truth (if you have one)?
- a canonical normal form (if we have one)?
- nothing in particular?

Research and tooling should think about (and clearly state) their metric and their reference.

CAN'T

7. Normal forms unlock the foundations.

What is a normal form?

A normal form picks **one unique representative** out of all expressions that compute the same mapping.

But **not every normal form is useful**:

- A **lookup table** is a normal form for any function. Trivially canonical, but completely useless.
- A **bit-level ANF** (XOR of ANDs) is a normal form. Algebraic leverage *per bit*, but not at the bit-vector level where MBA lives.

A useful normal form is one we can *do algebra* with.

Size of function classes through normal forms

A normal form gives a **unique representative per mapping**.

Thus, **counting normal forms = counting expressible mappings**.

We just did this for univariate polynomials: 2^{50} mappings on 8 bits.

Normal forms for MBA?

No general normal form is known for the different flavors of MBA.

Indeed, unrestricted MBA grammars are equivalent to **Universal** (all bit-vector functions).

Normal forms can be hybrid

A normal form doesn't have to be a **single** expression in a **single** grammar.

It can be **hybrid**: partition the input domain into chunks, use a normal form per chunk under a grammar that fits *there*.

This allows us to handle richer function classes as **piecewise instances of simpler ones**.

The theorem coming next is exactly this shape.

Theorem (Gàmez-Dresel) (piecewise-cubic). Every univariate T-function over $\mathbb{Z}/2^w\mathbb{Z}$ is a piecewise cubic polynomial mod 2^w on a regular partition of width 4.

(This is the worst case; in practice a coarser partition often works, with wider pieces and higher per-piece degree.)

T-functions become tractable through the polynomial machinery we already have.

What this looks like

$$f(x) = (x \vee 146)^2(x \wedge 91) \oplus 74 \quad \text{over } \mathbb{Z}/2^4\mathbb{Z}$$

Range	Piece polynomial $P(x)$
[0, 3]	$7x^3 + 14x^2 + 4x + 10$
[4, 7]	$7x^3 + 10x^2 + 12x + 10$
[8, 11]	$7x^3 + 6x^2 + 12x + 10$
[12, 15]	$7x^3 + 2x^2 + 4x + 10$

These polynomials are absolutely not random.

One class, several normal forms

A function class $\mathcal{A}$ can admit **several normal forms**:

- through different **grammars** that represent $\mathcal{A}$.
- through **hybrid constructions**.

CAN'T

8. Toward algebraic synthesis.

Univariate polynomial interpolation mod 2^w

Over $\mathbb{Z}$ or $\mathbb{R}$: $n + 1$ points define a unique polynomial (expression, and mapping) of degree $\leq n$ that passes through them.

Over $\mathbb{Z}/2^w\mathbb{Z}$: existence, uniqueness, and generic algorithms for polynomial interpolation mod 2^w are **not publicly solved**.

Polynomial Interpolation Modulo 2^w : Existence, Uniqueness, and Algorithms with Applications to Inversion of Binary Permutation Polynomials and Piecewise Interpolation. Gàmez-Montolio, Brain, Howe. TBA.

What about in general?

Polynomial interpolation handles polynomial mappings over $\mathbb{Z}/2^w\mathbb{Z}$. What about every other function class?

The natural generalization: pose interpolation for *any* (function class, normal form) pair. I'm calling this **algebraic synthesis**.

(There's no published literature under this name. It's a framing I'm proposing, not a field. Treat what follows as a research direction.)

Interpolation

samples of $f \rightarrow$ polynomial of f
= a **normal form** (under a given polynomial grammar)

Algebraic synthesis

samples of $f \rightarrow$ which class? $\rightarrow$ which normal form?
 $\rightarrow$ the unique representative optimizing a metric

The holy grail

If we know the **boundaries, counts, and at least one useful normal form** for each class we want to handle, then:

1. **Normalization** for many (class, normal form) pairs.
2. **From samples**, identify which classes the mapping can (or cannot) belong to.
3. **Build the normal form that optimizes the metric we care about.**

Find the best normal form, for the metric you care about, that you can actually compute with.

We have to bound the class (I)

Theoretically infeasible in general. Over a single variable, the space of mappings $\mathbb{Z}/2^w\mathbb{Z} \rightarrow \mathbb{Z}/2^w\mathbb{Z}$ already has size $(2^w)^{2^w}$. Over v variables: $(2^w)^{2^{vw}}$, much worse.

Algebraic synthesis on the whole space is hopeless.

If it weren't, every cryptosystem's I/O behavior could be expressed in "simple" normal forms.

We have to bound the class (II)

Probably practically infeasible even for some of the richer named classes we'd care about (the counts blow up fast with variables, depth, degree).

So we don't try in full generality.

In practice we *choose* what to bound: alphabet, depth, degree, bit-influence direction, input range. Each choice collapses the class.

How many samples do we need?

To select a representative of a class $\mathcal{A}$, we need enough samples to distinguish it from every other representative. Each sample contributes **at most w bits** of information.

$$\text{\#samples} \geq \frac{\log_2 |\mathcal{A}|}{w}$$

The count of normal forms gives us an **information-theoretic lower bound** on sample complexity.

Caveat. This is just a lower bound. It does *not* mean that an efficient (or even conceivable) algorithm that hits it exists; in practice we usually need more samples.

Example: 8-bit univariate polynomials give $50/8 \approx 7$ samples by the bound, but any general interpolation algorithm you can think of needs at least 10 (degree-9 polynomial: 10 coefficients to solve for).

Where this lands

For the slices where the math has caught up (polynomials mod 2^w), we start to have answers.

For the rest, we're still **making assumptions and hoping**.

The math is what tells us where we are, and where to look next.

What the field needs

- **State the slice.** Operator alphabet, rules of combination, function class, objective.
- **State the measurement.** Metric for "simpler", and reference to compare against.
- **Standardize datasets.** Build them outside the technique they evaluate. Binary corpora across compilers, archs, opt levels.
- **Use iterative loops,** not single-shot retrieval. Treat MBA simplification as one stage of a larger deobfuscation strategy.
- **Do the foundational work.** Useful normal forms, counts, clear definitions and boundaries across distinct functional classes. Pursue "algebraic synthesis" as a research direction.

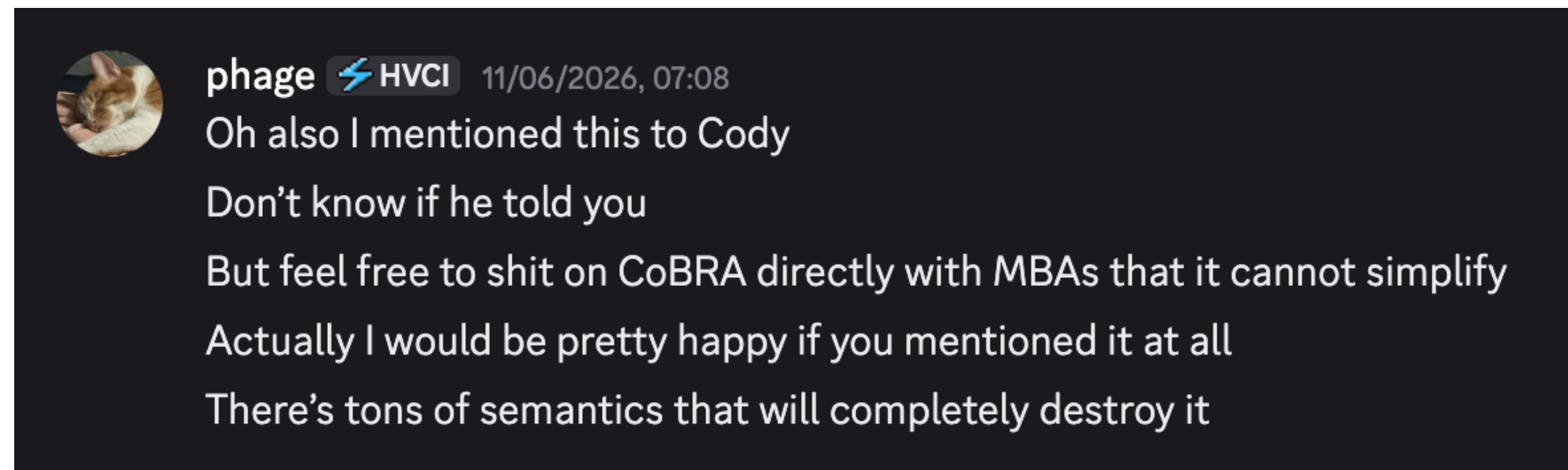
EXTRAS

Stress-testing CoBRA

CoBRA is the shiny new MBA simplifier, by **Kyle Elliott** at Trail of Bits.

Kyle's happy to see CoBRA broken on stage, so here it is.

(*phage* = Kyle on chat.)



<https://github.com/trailofbits/CoBRA>

<https://blog.trailofbits.com/2026/04/03/simplifying-mba-obfuscation-with-cobra/>

What follows are examples CoBRA can't handle. Grouped by failure mode.

Most courtesy of **Cody Dresel**.

STRESS-TESTING COBRA • UNSUPPORTED

CoBRA explicitly returns "unsupported" / refuses to process.

Non-constant right shifts

Original: `x`

Obfuscated: `x + ((x2 >> 7) >> (x3 | 1))`

```
[cobra-cli % ./cobra-cli --mba "x + ((x2 >> 7) >> (x3 | 1))"
Error: unsupported: shift amount must be an integer literal
```

Polynomials within bitwise sub-expressions

Original: $x + (x^4 \mid 10*y + 5)$

Obfuscated: $10*y + 5 + x + x^4 - (x^4 \& (10*y + 5))$

```
[cobra-cli % ./cobra-cli --mba "10*y+5 + x + x^4 - (x^4&(10*y+5))" ]  
Unsupported: semilinear normalization failed: expression is not semilinear (variable  
*variable multiplication or shift of non-bitwise operand)
```

STRESS-TESTING COBRA • NOT SIMPLIFIED

CoBRA accepts the input but can't reduce it.

Basic constant relation

Original:

```
x1 + x2
```

Obfuscated:

```
((((x1 + x2)*(1 + ((x3 & 1)*-2))) ^ ((x5 + 3)*(1 + ((x4 & 1)*-2))) ^ ((x5 + 3)*  
(1 + ((x4 & 1)*-2))))*(1 + ((x3 & 1)*-2)))
```

```
[cobra-cli % ./cobra-cli --mba "(((x1+x2) * ((1 + ((x3 & 1) * -2)))) ^ (((x5  
+ 3) * ((1 + ((x4 & 1) * -2)))) ^ (((x5 + 3) * ((1 + ((x4 & 1) * -2))))))  
* ((1 + ((x3 & 1) * -2))))"  
(x1 + x2) * ((x3 & 1) * -2 + 1) ^ (x5 + 3) * ((x4 & 1) * -2 + 1) ^ (x5 + 3)  
* ((x4 & 1) * -2 + 1) * ((x3 & 1) * -2 + 1)
```

Involution with right shifts

Original:

`x`

Obfuscated:

```
x ^ x>>5 ^ x>>7 ^ (x ^ x>>5 ^ x>>7)>>5 ^ (x ^ x>>5 ^ x>>7)>>7
```

```
[cobra-cli % ./cobra-cli --mba "x ^ x >> 5 ^ x >> 7 ^ (x ^ x >> 5 ^ x >> 7) >> 5 ^ (x  
^ x >> 5 ^ x >> 7) >> 7"  
x ^ x >> 5 ^ x >> 7 ^ (x ^ x >> 5 ^ x >> 7) >> 5 ^ (x ^ x >> 5 ^ x >> 7) >> 7
```


Masking detection failure

Original:

```
x & 128
```

Obfuscated:

```
((((x & 128) + (y & 127)) & 128))
```

```
[cobra-cli % ./cobra-cli --mba "(((x & 128) + (y & 127)) & 128))"  
Unsupported: Full-width verification failed  
(x & 128) + (y & 127) & 128
```


Involution cancellation failure

Original:

```
((x + y) & z) | 3
```

Obfuscated:

```
((((x + y) & z) | 3) ^ (((x*z) | (y & 85)) + w)) ^ (((x*z) | (y & 85)) + w)
```

```
[cobra-cli % ./cobra-cli --mba "(((x + y) & z) | 3) ^ ((x * z) | (y & 85)) + w) ^ ((x * z) | (y & 85)) + w)"
Unsupported: Full-width verification failed
(x + y & z | 3) ^ (x * z | y & 85) + w ^ (x * z | y & 85) + w
```

STRESS-TESTING COBRA • SUBOPTIMAL CHOICE OF METRIC / GRAMMAR

CoBRA returns something more complex than the input.

Linear MBA compact form vs. boolean basis reconstruction

Original:

$$x_0 + x_1 + x_2 + x_3 + x_4 + x_5 + x_6 + x_7 - (x_0 \wedge x_1 \wedge x_2 \wedge x_3 \wedge x_4 \wedge x_5 \wedge x_6 \wedge x_7)$$

```
cobra-cli % ./cobra-cli --mba "x0+x1+x2+x3+x4+x5+x6+x7 - (x0^x1^x2^x3^x4^x5^x6^x7)"
8 * (x4 & x5 & x6 & x7) + 2 * (x0 & (x1 ^ x2)) + 2 * (x1 & (x2 ^ x3)) + 2 * (x3 & (x0 ^ x2)) + 2 * (x4 & (x0 ^ x1)) + 2 * (x4 & (x2 ^ x3)) + 2 * (x5 & (x0 ^ x1)) + 2 * (x5 & (x2 ^ x3)) + 2 * (x4 & (x5 ^ x6)) + 2 * (x6 & (x0 ^ x1)) + 2 * (x6 & (x2 ^ x3)) + 2 * (x5 & (x6 ^ x7)) + 2 * (x7 & (x0 ^ x1)) + 2 * (x7 & (x2 ^ x3)) + 2 * (x7 & (x4 ^ x6)) + -4 * (x0 & x1 & x3 ^ x0 & x3 & x4) + -4 * (x4 & x2 & (x0 ^ x1)) + -4 * (x1 & x3 & x4 ^ x1 & x3 & x5) + -4 * (x5 & x2 & (x0 ^ x1)) + -4 * (x5 & x0 & (x3 ^ x4)) + -4 * (x5 & x4 & (x1 ^ x2)) + -4 * (x3 & x4 & x5 ^ x3 & x4 & x6) + -4 * (x6 & x2 & (x0 ^ x1)) + -4 * (x6 & x3 & (x0 ^ x1)) + -4 * (x6 & x4 & (x0 ^ x1)) + -4 * (x6 & x2 & (x4 ^ x5)) + -4 * (x6 & x5 & (x0 ^ x1)) + -4 * (x3 & x5 & x6 ^ x3 & x5 & x7) + -4 * (x7 & x2 & (x0 ^ x1)) + -4 * (x7 & x3 & (x0 ^ x1)) + -4 * (x7 & x4 & (x0 ^ x1)) + -4 * (x7 & x4 & (x2 ^ x3)) + -4 * (x7 & x5 & (x0 ^ x1)) + -4 * (x7 & x5 & (x2 ^ x4)) + -4 * (x7 & x6 & (x0 ^ x1)) + -4 * (x7 & x6 & (x2 ^ x3)) + 8 * (x0 & x1 & x2 & x3 ^ x0 & x2 & x3 & x4) + 8 * (x1 & x2 & x3 & x4 ^ x1 & x2 & x3 & x5) + 8 * (x5 & x3 & x0 & (x1 ^ x2)) + 8 * (x5 & x4 & x0 & (x1 ^ x2)) + 8 * (x2 & x3 & x4 & x5 ^ x2 & x3 & x4 & x6) + 8 * (x6 & x3 & x2 & (x0 ^ x1)) + 8 * (x6 & x4 & x2 & (x0 ^ x1)) + 8 * (x6 & x4 & x3 & (x0 ^ x1)) + 8 * (x6 & x5 & x2 & (x0 ^ x1)) + 8 * (x6 & x5 & x3 & (x0 ^ x1)) + 8 * (x2 & x3 & x5 & x6 ^ x2 & x3 & x5 & x7) + 8 * (x6 & x5 & x4 & (x0 ^ x1)) + 8 * (x7 & x3 & x2 & (x0 ^ x1)) + 8 * (x7 & x4 & x2 & (x0 ^ x1)) + 8 * (x7 & x4 & x3 & (x0 ^ x1)) + 8 * (x7 & x5 & x2 & (x0 ^ x1)) + 8 * (x7 & x5 & x3 & (x0 ^ x1)) + 8 * (x7 & x5 & x4 & (x0 ^ x1)) + 8 * (x7 & (x3 & x4 & x5 ^ x6 & x3 & x4)) + 8 * (x7 & x6 & x2 & (x0 ^ x1)) + 8 * (x7 & x6 & x3 & (x0 ^ x1)) + 8 * (x7 & x6 & x4 & (x0 ^ x1)) + 8 * (x7 & x6 & x2 & (x4 ^ x5)) + 8 * (x7 & x6 & x5 & (x0 ^ x1)) + -16 * (x0 & x1 & x3 & x4 & x5 ^ x0 & x2 & x3 & x4 & x5) + -16 * (x0 & x2 & x3 & x4 & x6 ^ x1 & x2 & x3 & x4 & x6) + -16 * (x0 & x2 & x3 & x5 & x6 ^ x1 & x2 & x3 & x5 & x6) + -16 * (x0 & x2 & x4 & x5 & x6 ^ x1 & x2 & x4 & x5 & x6) + -16 * (x0 & x3 & x4 & x5 & x6 ^ x1 & x3 & x4 & x5 & x6) + -16 * (x0 & x2 & x3 & x4 & x7 ^ x1 & x2 & x3 & x4 & x7) + -16 * (x0 & x2 & x3 & x5 & x7 ^ x1 & x2 & x3 & x5 & x7) + -16 * (x0 & x2 & x4 & x5 & x7 ^ x1 & x2 & x4 & x5 & x7) + -16 * (x0 & x3 & x4 & x5 & x7 ^ x1 & x3 & x4 & x5 & x7) + -16 * (x2 & x3 & x4 & x5 & x7 ^ x2 & x3 & x4 & x6 & x7) + -16 * (x0 & x2 & x3 & x6 & x7 ^ x1 & x2 & x3 & x6 & x7) + -16 * (x0 & x2 & x4 & x6 & x7 ^ x1 & x2 & x4 & x6 & x7) + -16 * (x0 & x3 & x4 & x6 & x7 ^ x1 & x3 & x4 & x6 & x7) + -16 * (x0 & x2 & x5 & x6 & x7 ^ x1 & x2 & x5 & x6 & x7) + -16 * (x0 & x3 & x5 & x6 & x7 ^ x1 & x3 & x5 & x6 & x7) + -16 * (x0 & x4 & x5 & x6 & x7 ^ x1 & x4 & x5 & x6 & x7) + 32 * (x0 & x2 & x3 & x4 & x5 & x6 ^ x1 & x2 & x3 & x4 & x5 & x6) + 32 * (x0 & x2 & x3 & x4 & x5 & x7 ^ x1 & x2 & x3 & x4 & x5 & x7) + 32 * (x0 & x2 & x3 & x4 & x6 & x7 ^ x1 & x2 & x3 & x4 & x6 & x7) + 32 * (x0 & x2 & x3 & x5 & x6 & x7 ^ x1 & x2 & x3 & x5 & x6 & x7) + 32 * (x0 & x2 & x4 & x5 & x6 & x7 ^ x1 & x2 & x4 & x5 & x6 & x7) + 32 * (x0 & x3 & x4 & x5 & x6 & x7 ^ x1 & x3 & x4 & x5 & x6 & x7) + -64 * (x0 & x2 & x3 & x4 & x5 & x6 & x7 ^ x1 & x2 & x3 & x4 & x5 & x6 & x7)
```


Polynomial blow-up over 16 bits

Original:

$$(x - 1)^{100} \text{ over } \mathbb{Z}/2^{16}\mathbb{Z}$$

```
[cobra-cli % ./cobra-cli --mba "(x-1)**100" --bitwidth 16  
-~(24476 * x + -25770 * x * x + -15012 * x * x * x + 15049 * x * x * x * x + 23792 *  
  x * x * x * x * x + -2544 * x * x * x * x * x * x * x + -29144 * x * x * x * x * x * x * x  
* x + 5396 * x * x * x * x * x * x * x * x * x + -10963 * x * x * x * x * x * x * x * x * x  
* x + 5170 * x * x * x * x * x * x * x * x * x * x * x * x + -15432 * x * x * x * x * x * x * x * x  
* x * x * x * x * x * x * x + 24993 * x * x * x * x * x * x * x * x * x * x * x * x * x * x * x * x * x + -18866  
  * x * x * x * x * x * x * x * x * x * x * x * x * x * x * x * x * x * x + 10608 * x * x * x * x * x * x * x * x  
  * x * x * x * x * x * x * x * x * x * x * x * x + 8380 * x * x * x * x * x * x * x * x * x * x * x * x * x * x  
* x * x * x * x * x * x + -135 * x * x * x * x * x * x * x * x * x * x * x * x * x * x * x * x * x * x * x * x * x * x  
  x + x * x * x * x * x * x * x * x * x * x * x * x * x * x * x * x * x * x * x * x * x * x)
```

Thank you

Arnau Gàmez i Montolio

arnaugamez.com

Questions, disagreements, and counter-examples especially welcome.